



JavaScript Fundamental...



A 1.20 ▾

|A| 0% ▾



2. Function Expression:

```
const sayHello = function(name) {
  return 'Hello, ' + name + '!';
};
console.log(sayHello('Bob')); // Output: Hello, Bob!
```

3. Arrow Function (ES5): Concise syntax, especially for anonymous functions.

```
const multiply = (a, b) => a * b;
console.log(multiply(4, 5)); // Output: 20

const sayHi = name => 'Hi, ' + name!';
console.log(sayHi('Charlie')); // Output: Hi, Charlie!
```

Hindi: फ़ंक्शन कोई के स्क्रिप्ट होता है जिन्हें एक विशेष काम करने के लिए डिज़ाइन किया जाता है। उन्हें कई तरीकों से परिभाषित किया जा सकता है।

1. फ़ंक्शन घोषणा (Function Declaration):

```
function greet(name) {
  return 'Hello, ' + name + '!';
}
console.log(greet('Alice')); // आउटपुट: Hello, Alice!
```

2. फ़ंक्शन एक्सप्रेशन (Function Expression):

```
const sayHello = function(name) {
  return 'Hello, ' + name + '!';
};
console.log(sayHello('Bob')); // आउटपुट: Hello, Bob!
```

एरो फ़ंक्शन (Arrow Function) (ES6): संक्षिप्त सिंटैक्स, विशेष रूप से अनॉनस फ़ंक्शन के लिए।

```
const multiply = (a, b) => a * b;
console.log(multiply(4, 5)); // आउटपुट: 20

const sayHi = name => 'Hi, ' + name!';
console.log(sayHi('Charlie')); // आउटपुट: Hi, Charlie!
```

What is Callback Hell? (कॉलबैक हेल क्या है?)

Callback hell, also known as "Pyramid of Doom," is a situation in asynchronous JavaScript programming where multiple nested callback functions make the code difficult to read, understand, and maintain. It often arises when handling a sequence of asynchronous operations that depend on each other.

Example (Illustrative of the problem):

```
// This is an oversimplified example to show nesting
getData(function(a) {
  getMoreData(a, function(b) {
    getEvenMoreData(b, function(c) {
      console.log("Finally got all data:", a, b, c);
    });
  });
});
```

Solution (using Promises/Async-Await):

```
// Using Promises
getData()
  .then(a => getMoreData(a))
  .then(b => getEvenMoreData(b))
  .then(c => console.log("Finally got all data:", c))
  .catch(error => console.error(error));

// Using Async/Await (more readable)
async function fetchData() {
  try {
    const a = await getData();
    const b = await getMoreData(a);
    const c = await getEvenMoreData(b);
    console.log("Finally got all data:", c);
  } catch (error) {
    console.error(error);
  }
}
```

Hindi: कॉलबैक हेल, जिसे 'पायramid ऑफ़ डूम' के रूप में भी जाना जाता है, एक्सिक्यूटिंग JavaScript प्रोग्रामिंग में एक ऐसी स्थिति है जहाँ कई नेस्टेड कॉलबैक फ़ंक्शन कोड को पढ़ने, समझने और बनाए रखने में कठिनाई उत्पन्न करता है। यह अक्सर एक से अधिक असिंक्रोनस ऑपरेशन्स के एक अनुक्रम को संभालते समय उत्पन्न होता है जो एक दूसरे पर निर्भर करते हैं।

उदाहरण (समस्या को दर्शाने वाले):

```
// एक सरल उदाहरण के लिए एक असिंक्रोनस उदाहरण है
getData(function(a) {
  getMoreData(a, function(b) {
    getEvenMoreData(b, function(c) {
      console.log("Finally got all data:", a, b, c);
    });
  });
});
```

समाधान (प्रॉमिस/एसैन्क-अवेट का उपयोग करके):

```
// प्रॉमिस का उपयोग करके
getData()
  .then(a => getMoreData(a))
  .then(b => getEvenMoreData(b))
  .then(c => console.log("Finally got all data:", c))
  .catch(error => console.error(error));
```

```
cul id="myList">
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>
<script>
  const list =
    document.getElementById('myList');

  // जबकि कि क्या क्लिक किया गया एलिमेंट एक <li> है
  if (event.target.tagName === 'LI') {
    console.log('Clicked item:',
      event.target.textContent);
  });

  // मौजूदा रूप से एक नया एलिमेंट जोड़ें
  const newItem = document.createElement('li');
  newItem.textContent = 'Item 4 (new)';
  list.appendChild(newItem);

  </script>
```

```
const person = {
  firstName: 'John',
  lastName: 'Doe',
  age: 30,
  greet: function() {
    console.log('Hello, ' +
      `${this.firstName}`);
  };
  console.log(person.firstName); // John
  person.greet(); // Hello, John

2. new Object() constructor:
const car = new Object();
car.make = 'Toyota';
car.model = 'Camry';
console.log(car.make); // Toyota

3. Constructor Function:
function Animal(name, species) {
  this.name = name;
  this.species = species;
  this.makeSound = function() {
    console.log(`${this.name} makes a sound.`);
  };
};

const dog = new Animal('Buddy', "Dog");
console.log(dog.species); // Dog
dog.makeSound(); // Buddy makes a sound.
```

How to Create an Object in JavaScript (JavaScript में एक ऑब्जेक्ट कैसे बनाएं)

Objects in JavaScript are collections of key-value pairs. They can be created in several ways:

```
1. Object Literal (most common):
const person = {
  firstName: 'John',
  lastName: 'Doe',
  age: 30,
  greet: function() {
    console.log('Hello, ' +
      `${this.firstName}`);
  };
  console.log(person.firstName); // John
  person.greet(); // Hello, John

2. new Object() constructor:
const car = new Object();
car.make = 'Toyota';
car.model = 'Camry';
console.log(car.make); // Toyota

3. Constructor Function:
function Animal(name, species) {
  this.name = name;
  this.species = species;
  this.makeSound = function() {
    console.log(`${this.name} makes a sound.`);
  };
};

const dog = new Animal('Buddy', "Dog");
console.log(dog.species); // Dog
dog.makeSound(); // Buddy makes a sound.
```

```
// संक्षेप/अवर का उपयोग करके (अनॉनस फ़ंक्शन)
async function fetchData() {
  try {
    const a = await getData();
    const b = await getMoreData(a);
    const c = await getEvenMoreData(b);
    console.log("Finally got all data:", c);
  } catch (error) {
    console.error(error);
  }
}
```

What is an Anonymous Function? (एक अनॉनस फ़ंक्शन क्या है?)

An anonymous function is a function that does not have a name. It is typically used as a function expression or as a callback function, where it is passed as an argument to another function.

Example:

```
// Function expression (anonymous)
const greet = function() {
  console.log('Hello from an anonymous function!');
}();

// Anonymous function as a callback
setTimeout(function() {
  console.log("This executed after 1 second (anonymous callback).");
}, 1000);

// Anonymous arrow function
const multiply = (a, b) => a * b;
console.log(multiply(3, 7));
```

Hindi: एक अनॉनस फ़ंक्शन एक ऐसा फ़ंक्शन है जिसका कोई नाम नहीं होता है। इसका उपयोग आमतौर पर एक फ़ंक्शन एक्सप्रेशन के रूप में या एक कॉलबैक फ़ंक्शन के रूप में किया जाता है, जहाँ इसे दूसरे फ़ंक्शन में एक आर्गुमेंट के रूप में पास किया जाता है।

उदाहरण:

```
// फ़ंक्शन एक्सप्रेशन (अनॉनस)
const greet = function() {
  console.log('Hello from an anonymous function!');
}();

// कॉलबैक के रूप में अनॉनस फ़ंक्शन
setTimeout(function() {
  console.log("This executed after 1 second (anonymous callback).");
}, 1000);

// अनॉनस एरो फ़ंक्शन
const multiply = (a, b) => a * b;
console.log(multiply(3, 7));
```

What is the this keyword in JavaScript? (JavaScript में this कीवर्ड क्या है?)

The this keyword in JavaScript refers to the object it belongs to. Its value depends on how the function is called.

- Global context:** this refers to the global object (window in browsers, global in Node.js).
- Method call:** this refers to the object that owns the method.

- Simple function call:** this refers to the global object (in non-strict mode) or undefined (in strict mode).
- Event handler:** this refers to the element that received the event.

- Arrow functions:** this retains the this value of the enclosing lexical context (it does not bind its own this).
- call(), apply(), bind():** Explicitly set the value of this.

Example:

```
const person = {
  name: 'John',
  greet: function() {
    console.log('Hello, my name is ' +
      `${this.name}`); // 'this' refers to 'person'
  }
};

person.greet(); // Output: Hello, my name is John

function showThis() {
  console.log(this);
}

showThis(); // In browser, 'this' is window object; in strict mode, undefined

const anotherPerson = {
  name: 'Jane',
  greetNow: () => {
    console.log('Hello, my name is ' +
      `${this.name}`); // 'this' refers to global object (or undefined in strict mode script)
  }
};

anotherPerson.greetNow(); // Output: Hello, my name is undefined (if in browser global scope, or empty if module scope)
```

Hindi: JavaScript में this कीवर्ड उस ऑब्जेक्ट को संदर्भित करता है जिससे यह संबद्ध है। इसका मान इस बात पर निर्भर करता है कि

फ़ंक्शन को कैसे कॉल किया जाता है।

- ग्लोबल संदर्भ:** this ग्लोबल ऑब्जेक्ट को संदर्भित करता है (ब्राउज़र में window, Node.js में global)।
- मैथड कॉल:** this उस ऑब्जेक्ट को संदर्भित करता है जिसके पास मैथड है।

- सिंपल फ़ंक्शन कॉल:** this ग्लोबल ऑब्जेक्ट को संदर्भित करता है (री-स्ट्रिक्ट मोड में) या undefined (स्ट्रिक्ट मोड में)।
- इवेंट हैंडलर:** this उस एलिमेंट को संदर्भित करता है जिसने इवेंट ट्राइगर किया है।

- एरो फ़ंक्शन:** this संदर्भित ऑब्जेक्ट संदर्भ के this मान को संभालता रहता है (जैसे अनॉनस this बॉन्ड नहीं करता है)।

- call(), apply(), bind():** this का मान स्पष्ट रूप से सेट करें।

उदाहरण:

```
const person = {
  name: 'John',
  greet: function() {
    console.log('Hello, my name is ' +
      `${this.name}`); // 'this' 'person' को संदर्भित करता है
  }
};

person.greet(); // आउटपुट: Hello, my name is John

function showThis() {
  console.log(this);
}

showThis(); // ब्राउज़र में, 'this' विंडो ऑब्जेक्ट है; स्ट्रिक्ट मोड में, undefined

const anotherPerson = {
  name: 'Jane',
  greetNow: () => {
    console.log('Hello, my name is ' +
      `${this.name}`); // 'this' ग्लोबल ऑब्जेक्ट को संदर्भित करता है (या स्ट्रिक्ट मोड में undefined)
  }
};

anotherPerson.greetNow(); // आउटपुट: Hello, my name is undefined (ब्राउज़र ग्लोबल स्कोप में है, या स्ट्रिक्ट मोड में undefined)
```

Explain Event Delegation (इवेंट डेलीगेशन समझिए)

Event delegation is a technique where you attach a single event listener to a parent element, instead of attaching individual listeners to multiple child elements. When an event is triggered on a child element, it "bubbles up" to the parent, allowing the single listener on the parent to catch and handle the event.

Benefits:

- Performance:** Fewer event listeners mean less memory consumption and better performance.
- Dynamic Elements:** Works automatically for elements added to the DOM dynamically after the page has loaded.
- Cleaner Code:** Reduces repetitive code.

Example:

```
<ul id="myList">
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>
<script>
  const list =
    document.getElementById('myList');

  list.addEventListener('click', function(event) {
    // Check if the clicked element is an <li>
    if (event.target.tagName === 'LI') {
      console.log('Clicked item:',
        event.target.textContent);
    });

    // Add a new item dynamically
    const newItem = document.createElement('li');
    newItem.textContent = 'Item 4 (new)';
    list.appendChild(newItem);
  });
</script>
```

Hindi: इवेंट डेलीगेशन एक ऐसी तकनीक है जहाँ आप कई बटन/लिंक्स पर एक ही एलिमेंट लिस्नर अटैच करते हैं बजाय, एक इवेंट एलिमेंट पर एक डिफरेंट इवेंट लिस्नर अटैच करने हैं। जब किसी घातल एलिमेंट पर कोई इवेंट ट्रिगर होता है, तो यह पैरेंट तक "बुलबुल" करता है, जिससे पैरेंट पर सिंगल लिस्नर इवेंट को पकड़ने और संभालने को अनुमति देता है।

फायदे:

- प्रदर्शन:** कम इवेंट लिस्नर का मतलब कम मेमोरी खपत और बेहतर प्रदर्शन।
- डायनामिक एलिमेंट्स:** पेज लोड होने के बाद DOM में परिवर्तित रूप से जोड़े गए एलिमेंट्स के लिए सिंगल लिस्नर काम करता है।

- स्वीटर कोड:** दोहराव वाले कोड को कम करता है।

उदाहरण:

```
• undefined:
  • A variable that has been declared but not yet assigned a value.
  • A function that doesn't explicitly return anything returns undefined.
  • A property that doesn't exist on an object.
  • The default value of function parameters if not provided.
  • 'Type: typeof undefined is 'undefined' .

• null:
  • Represents the intentional absence of any object value. It's a primitive value that indicates "no value," "nothing," or "empty."
  • It must be explicitly assigned.
  • 'Type: typeof null is 'object' (this is a long-standing bug in JavaScript, but it's typically treated as a primitive).

Example:
let a; // Declared but not assigned, so 'a' is undefined
console.log(a); // null

let b = null; // Explicitly assigned null
console.log(b); // null

console.log(a == b); // true (loose equality, both are "false")
console.log(a === b); // false (strict equality, different types)

console.log(typeof a); // "undefined"
console.log(typeof b); // "object" (historical bug)

const obj = {};
console.log(obj.nonexistentProperty); // undefined

Hindi: null और undefined दोनों एक मान की अनुपस्थिति को दर्शाते हैं, लेकिन वे अलग-अलग बातें दर्शाते हैं:
• undefined:
  • एक वेरिएबल जिसे घोषित किया गया है लेकिन अभी तक कोई मान असाइन नहीं किया गया है।
  • एक फ़ंक्शन जो स्पष्ट रूप से कुछ भी नहीं लौटाता है वह undefined लौटाता है।
  • एक पैरामीटर जो किसी ऑब्जेक्ट पर मौजूद नहीं है।
  • यदि प्रदान नहीं किया गया तो फ़ंक्शन मापदंडों का डिफ़ॉल्ट मान।
  • प्रकार: typeof undefined 'undefined' है।
• null:
  • किसी भी ऑब्जेक्ट मान की अनुपस्थिति को दर्शाता है।
  • एक स्पष्ट निर्दिष्ट मान है जो 'कोई मान नहीं' 'कुछ नहीं' या 'खाली' इंगित करता है।
  • इसे स्पष्ट रूप से असाइन किया जाना चाहिए।
  • प्रकार: typeof null 'object' है (यह JavaScript में एक लंबे समय से पैरेंट आ रहा मान है, लेकिन इसे आमतौर पर एक प्रिमाटिव के रूप में माना जाता है)।
```

उदाहरण:

```
const obj = {
  // ...
};

console.log(typeof obj); // "object" (ऐतिहासिक बग)

const obj = {};
```

Edit with chat

```
Tolkien";
book.getInfo(); // The Hobbit by J.R.R. Tolkien
```

होता है जिससे अन्य ऑब्जेक्ट गुण और व्यवहार को इन्हार करत है। इस वेरिएबल को प्रोटोटाइप चैन से इन्हार के रूप में जाना जाता है। जब आप किसी ऑब्जेक्ट पर किसी प्रॉपर्टी या मैथड टुक पहुंचने का

```
console.log(typeof obj); // "undefined"
console.log(typeof obj); // "object" (ऐतिहासिक बग)

const obj = {};
```